

Практическое задание: Развертывание ML-сервиса в Kubernetes

Цель: Закрепить навыки работы с Kubernetes на практике

Задание

Вам необходимо развернуть простой ML-сервис для предсказания цен на недвижимость в Kubernetes и настроить автоматическое масштабирование.

Что нужно сделать

Часть 1: Подготовка приложения

Задача 1.1: Создайте простое FastAPI приложение

Создайте файл `app.py` с простым ML-сервисом:

```
from fastapi import FastAPI
from pydantic import BaseModel
import random
import time

app = FastAPI(title="House Price Predictor")

class HouseData(BaseModel):
    square_meters: float
    rooms: int
    year_built: int

class PredictionResponse(BaseModel):
    predicted_price: float
    currency: str = "USD"

@app.get("/health")
def health():
    return {"status": "healthy"}

@app.get("/ready")
def ready():
    # Имитируем проверку готовности
    return {"status": "ready"}
```

```

@app.post("/predict", response_model=PredictionResponse)
def predict(data: HouseData):
    # Простая формула для предсказания (для демонстрации)
    # В реальности здесь была бы обученная ML модель
    base_price = 1000 # цена за квадратный метр
    age_factor = max(0.5, 1 - (2024 - data.year_built) * 0.01)
    room_bonus = data.rooms * 5000

    predicted_price = (data.square_meters * base_price * age_factor) +
    room_bonus

    # Имитируем задержку обработки
    time.sleep(0.1)

    return PredictionResponse(predicted_price=round(predicted_price, 2))

@app.get("/")
def root():
    return {"service": "House Price Predictor", "version": "1.0.0"}

```

Создайте requirements.txt:

```

fastapi==0.104.1
uvicorn[standard]==0.24.0
pydantic==2.5.0

```

Создайте Dockerfile:

```

FROM python:3.11-slim

```

```

WORKDIR /app

```

```

COPY requirements.txt .

```

```

RUN pip install --no-cache-dir -r requirements.txt

```

```

COPY app.py .

```

```

EXPOSE 8000

```

```

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

Задача 1.2: Соберите и опубликуйте Docker образ

Соберите образ (замените YOUR_USERNAME на ваш Docker Hub username)

```

docker build -t YOUR_USERNAME/house-price-api:v1 .

```

Протестируйте локально

```

docker run -d -p 8000:8000 YOUR_USERNAME/house-price-api:v1

```

Проверьте

```
curl http://localhost:8000/health
```

Остановите тестовый контейнер

```
docker stop $(docker ps -q --filter ancestor=YOUR_USERNAME/house-price-api:v1)
```

Опубликуйте в Docker Hub

```
docker login
```

```
docker push YOUR_USERNAME/house-price-api:v1
```

Часть 2: Развертывание в Kubernetes

Задача 2.1: Запустите Minikube

```
minikube start --memory=4096 --cpus=2
```

```
minikube addons enable metrics-server
```

Задача 2.2: Создайте Kubernetes манифесты

Создайте файл deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: house-price-api
  labels:
    app: house-price-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: house-price-api
  template:
    metadata:
      labels:
        app: house-price-api
    spec:
      containers:
        - name: api
          image: YOUR_USERNAME/house-price-api:v1 # ЗАМЕНИТЕ!
          ports:
            - containerPort: 8000
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "200m"
```

```

    livenessProbe:
      httpGet:
        path: /health
        port: 8000
      initialDelaySeconds: 10
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /ready
        port: 8000
      initialDelaySeconds: 5
      periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: house-price-api
spec:
  type: NodePort
  selector:
    app: house-price-api
  ports:
    - port: 8000
      targetPort: 8000
      nodePort: 30100

```

Задача 2.3: Разверните приложение

Примените манифест

```
kubectl apply -f deployment.yaml
```

Проверьте статус

```
kubectl get pods
```

```
kubectl get svc
```

Дождитесь, пока все поды будут Running

```
kubectl get pods -w
```

Получите URL

```
minikube service house-price-api --url
```

Сохраните URL в переменную

```
export API_URL=$(minikube service house-price-api --url)
```

Задача 2.4: Протестируйте API

Health check

```
curl $API_URL/health
```

Тестовое предсказание

```
curl -X POST $API_URL/predict \
```

Должны получить предсказание цены

Задача 3.1: Создайте HorizontalPodAutoscaler

```
while true; do
  wget -q -O- --post-data='{ "square_meters":100,"rooms":3,"year_built":2010}'
```

```
--header="Content-Type: application/json" \  
http://house-price-api:8000/predict  
done
```

В другом терминале наблюдайте за масштабированием:

```
# Терминал 2: следите за подами  
watch kubectl get pods
```

```
# Терминал 3: следите за HPA  
watch kubectl get hpa
```

```
# Терминал 4: следите за метриками  
watch kubectl top pods
```

После появления 3-5 подов, остановите нагрузку (Ctrl+C в терминале с load-generator)

Часть 4: Эксперименты

Задача 4.1: Протестируйте self-healing

```
# Найдите один из подов  
kubectl get pods
```

```
# Удалите под  
kubectl delete pod <имя-пода>
```

```
# Сразу проверьте - Kubernetes создаст новый  
kubectl get pods -w
```

Задача 4.2: Проверьте, что API продолжает работать

```
# Во время замены пода проверьте доступность
```

```
for i in {1..10}; do  
    curl -s $API_URL/health | jq .status  
    sleep 1  
done
```

Отчет

Создайте файл report.md и ответьте на следующие вопросы:

1. Основные наблюдения (2-3 предложения на вопрос)

Q1: Сколько времени потребовалось для создания новых подов при масштабировании?

Q2: Что произошло с API во время удаления пода? Были ли ошибки в запросах?

Q3: Сколько подов создал HPA под нагрузкой? При каком уровне CPU?

Q4: Сколько времени HPA ждал перед уменьшением количества подов после снятия нагрузки?

2. Сравнение с Docker Compose (3-4 предложения)

Q5: В чем основные преимущества Kubernetes перед Docker Compose для этого приложения?

Q6: Какие задачи были проще в Docker Compose, а какие - в Kubernetes?

3. Применение в ML (2-3 предложения)

Q7: Как бы вы использовали Kubernetes для реального ML-проекта? Какие компоненты развернули бы?

Q8: Какие проблемы ML-систем решает Kubernetes? (подсказка: вспомните лекции про построение архитектуры обработки данных и про транспортный слой)

Критерии оценки

Обязательная часть (80%):

-  **Приложение работает** (20%): API отвечает на запросы, health checks работают
-  **Развернуто в Kubernetes** (20%): используются Deployment и Service
-  **Настроено автомасштабирование** (15%): HPA работает и реагирует на нагрузку
-  **Протестирован self-healing** (15%): поды автоматически восстанавливаются
-  **Скриншоты/запись** (10%): визуальное подтверждение работы

Дополнительные баллы (20%):

-  **Отчет с наблюдениями** (15%): детальные ответы на вопросы
-  **Использованы resource limits** (5%): правильно настроены requests и limits

Задания повышенной сложности (опционально)

Если у вас осталось время и хочется углубиться:

Задание A: Rolling Update

1. Измените app.py - добавьте version: "2.0.0" в ответ root эндпоинта

2. Соберите новый образ `YOUR_USERNAME/house-price-api:v2`
3. Выполните rolling update:

```
kubectl set image deployment/house-price-api api=YOUR_USERNAME/house-price-api:v2
```
4. Наблюдайте за процессом: `kubectl rollout status deployment/house-price-api`
5. Проверьте, что API не был недоступен во время обновления

Задание Б: Добавьте Redis для кэширования

1. Разверните Redis в кластере
2. Модифицируйте `app.py` для кэширования предсказаний
3. Обновите deployment с переменными окружения для подключения к Redis

Задание В: Namespaces для разных окружений

1. Создайте два namespace: `dev` и `prod`
2. Разверните приложение в обоих с разными конфигурациями
3. Покажите изоляцию между окружениями

Что сдавать

Создайте ZIP архив `kubernetes_homework_<ФАМИЛИЯ>.zip` со следующими файлами:

```
kubernetes_homework/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
├── deployment.yaml  
├── hpa.yaml  
├── report.md (опционально)  
├── screenshots/  
│   ├── pods_running.png  
│   ├── hpa_scaling.png  
│   └── load_test.png
```



Подсказки и частые ошибки

Ошибка: ImagePullBackOff

Причина: Kubernetes не может скачать образ

Решение:

- Проверьте, что образ опубликован: `docker search YOUR_USERNAME/house-price-api`
- Проверьте имя образа в `deployment.yaml`
- Убедитесь, что образ публичный (не private)

Ошибка: `CrashLoopBackOff`

Причина: Приложение падает при запуске

Решение:

- Посмотрите логи: `kubectl logs <pod-name>`
- Проверьте, что приложение работает локально в Docker

HPA не масштабирует

Причина: Metrics server не работает или нет нагрузки

Решение:

- Проверьте: `kubectl top pods`
- Включите metrics: `minikube addons enable metrics-server`
- Подождите 1-2 минуты после создания HPA
- Убедитесь, что нагрузка достаточная

Не могу получить доступ к сервису

Причина: Service неправильно настроен

Решение:

- Проверьте: `kubectl get svc`
- Используйте: `minikube service house-price-api --url`
- Альтернатива: `kubectl port-forward svc/house-price-api 8000:8000`

Поды не становятся Ready

Причина: Readiness probe не проходит

Решение:

- Проверьте логи: `kubectl logs <pod-name>`
 - Проверьте: `kubectl describe pod <pod-name>`
 - Убедитесь, что `/ready` endpoint работает локально
-

Полезные команды для отладки

Общая информация о кластере

```
kubectl cluster-info
```

```
kubectl get nodes
```

Статус ресурсов

```
kubectl get all
```

```
kubectl get pods -o wide
```

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Детальная информация

```
kubectl describe pod <pod-name>
```

```
kubectl describe deployment house-price-api
```

```
kubectl describe hpa house-price-hpa
```

Логи

```
kubectl logs <pod-name>
```

```
kubectl logs -f <pod-name> # follow mode
```

```
kubectl logs <pod-name> --previous # логи предыдущего контейнера
```

Выполнение команд в поде

```
kubectl exec <pod-name> -- env
```

```
kubectl exec -it <pod-name> -- /bin/sh
```

Удаление ресурсов

```
kubectl delete deployment house-price-api
```

```
kubectl delete svc house-price-api
```

```
kubectl delete hpa house-price-hpa
```

Или всё сразу:

```
kubectl delete -f deployment.yaml
```

```
kubectl delete -f hpa.yaml
```

Полная очистка

```
minikube delete # удаляет весь кластер
```